

AI-Powered Employee Productivity Analytics and Burnout Prediction System

Major Project Report

Programme: Master of Computer Applications (MCA)

Subject Code: 23ONMCR-753

Institution: Chandigarh University **Centre:** Centre for Distance & Online Education

Batch: Jul 2023

Title Page

Project Title: AI-Powered Employee Productivity Analytics and Burnout Prediction System

Submitted By: *[Student Name]*

Roll No.: *[Roll Number]*

University Roll No.: *[University Roll Number]*

Supervised By: *[Guide Name]*

Submitted To: Chandigarh University

Certificate

This is to certify that the project titled **AI-Powered Employee Productivity Analytics and Burnout Prediction System** is a bonafide record of the work carried out by **[Student Name]** in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) under Chandigarh University.

Project Guide: _____

Department/Coordinator: _____

Date: _____

Place: _____

Declaration

I hereby declare that the project report entitled **AI-Powered Employee Productivity Analytics and Burnout Prediction System** is an original work carried out by me under the guidance of my project supervisor. This report has not been submitted earlier, in full or in part, for the award of any degree or diploma.

Student Name: _____

Signature: _____

Date: _____

Acknowledgement

I express my sincere gratitude to Chandigarh University for providing me the opportunity to undertake this major project as part of the MCA curriculum. I am thankful to my project guide for the valuable support, suggestions, and guidance throughout the development of this project.

I also acknowledge the support of my teachers, friends, and family members who encouraged me during the project work. This project helped me improve my understanding of full-stack development, database design, role-based security, and machine learning integration in enterprise applications.

Abstract

Employee productivity and burnout are critical concerns in modern organizations. Excessive workload, long working hours, poor task distribution, and low work-life balance can reduce productivity and increase the risk of burnout. Manual monitoring of such conditions is often inconsistent and reactive.

This project presents an AI-powered employee productivity analytics and burnout prediction system designed to support HR departments, managers, and employees through role-based access and intelligent insights. The system provides secure login, employee management, department tracking, manager-wise reportee access, worklog capture, HR analytics, and burnout prediction support.

The application is implemented using Angular for the frontend, FastAPI for the backend, SQLAlchemy with SQLite for persistence in development, and Python-based machine learning utilities for predictive analytics. The project demonstrates how software engineering, database design, and machine learning can be combined into a practical business solution for workforce monitoring and decision support.

Table of Contents

1. Introduction
 2. Objectives
 3. Scope of the Project
 4. Hardware and Software Requirements
 5. SDLC of the Project
 6. Design
 7. Coding & Implementation
 8. Testing
 9. Application
 10. Conclusion
 11. Bibliography
-

1. Introduction

Organizations require reliable tools to understand productivity patterns, reduce burnout risk, and improve employee engagement. Traditional manual methods are usually slow and do not provide timely insights. The proposed system addresses these limitations by offering a web-based platform that supports employee productivity analytics and burnout prediction.

The system is designed as an enterprise-style application with separate roles for Employee, Manager, and HR. Each role sees only the data relevant to its responsibilities. The platform supports authentication, employee data management, manager reportee views, HR-level monitoring, worklogs, and machine learning-driven analytics.

1.1 Problem Statement

Organizations often fail to identify early burnout signals such as heavy workload, excessive overtime, insufficient rest, or poor attendance patterns. Without a centralized system, managers and HR teams cannot easily analyze employee activity and intervene at the right time.

1.2 Proposed Solution

The proposed solution is a secure full-stack application that stores employee data, captures work activity, and uses ML-based analytics to support productivity monitoring and burnout prediction.

1.3 Objectives

- Build a role-based employee productivity analytics platform.
- Provide secure login and authorization.

- Manage employee and department information.
- Allow managers to view their reportees.
- Allow HR to view organization-level employee data.
- Support worklog capture and monitoring.
- Support machine learning-based burnout prediction.
- Improve decision-making through analytics and reports.

1.4 Broad Area of Application

The project belongs to the following broad application areas: - Machine Learning
- Software Engineering - Database Management Systems - Web Application Development

1.5 Hardware Requirements

The project can run on a standard development workstation. - Processor: Intel i5 / Ryzen 5 or above - RAM: 8 GB or more recommended - Storage: 256 GB SSD or above - Internet access for package installation and updates - Standard keyboard, mouse, and display interfaces

1.6 Software Requirements

- Frontend: Angular
- Backend: FastAPI (Python)
- ORM: SQLAlchemy
- Database: SQLite for development; configurable for MySQL/PostgreSQL
- ML: Scikit-Learn, Joblib, Pandas, NumPy
- IDE: Visual Studio Code
- Version Control: Git
- Operating System: Windows, macOS, or Linux

1.7 Scope of the Project

The project is intended for organizations that want a practical web-based solution for employee productivity monitoring and burnout awareness. The current scope includes secure authentication, employee data access, manager-wise reportee visibility, HR-level employee review, worklog handling, and burnout-oriented analytics.

The system is suitable for internal organizational use in HR departments, team management, and employee wellness monitoring. The modular architecture also supports future expansion into attendance, leave management, task tracking, and advanced reporting features.

1.8 Project Synopsis

Title: AI-Powered Employee Productivity Analytics and Burnout Prediction System

Objective: To build a secure and intelligent HR analytics platform that can monitor employee activity, support managerial decision-making, and predict burnout risk.

Resources Required: - Angular frontend for user interface - FastAPI backend for APIs and business logic - SQLAlchemy-based database layer - Python machine learning utilities for analytics - Development tools such as VS Code and Git

2. SDLC of the Project

The project follows a structured Software Development Life Cycle to ensure clarity, maintainability, and quality.

2.1 Requirement Analysis

In this phase, the project requirements were identified from the problem domain of HR analytics and employee burnout prediction. The requirements included secure login, employee data handling, managerial visibility, HR analytics, worklog support, and prediction services.

2.2 Planning

The project was planned as a modular web application with separate frontend, backend, database, and ML components. Role-based access and reusable API services were included in the planning stage to keep the architecture maintainable.

2.3 Design

The design phase defined the system architecture, database relationships, API structure, user roles, UI flow, and prediction pipeline. A clean separation between presentation, business logic, persistence, and ML processing was adopted.

2.4 Development

The development phase involved implementation of FastAPI backend services, SQLAlchemy models, Pydantic schemas, JWT authentication, Angular frontend components, and machine learning utilities.

2.5 Testing

The system was validated through API testing, role-based access checks, login verification, worklog submission checks, and data flow validation across modules.

2.6 Deployment

The application is designed to run in a local development environment using a Python backend and Angular frontend. The backend serves the APIs, and the frontend consumes them over HTTP.

2.7 Maintenance

The system is structured so future enhancements can be added with minimal disruption. Examples include stronger route guards, additional reports, pagination, and improved ML training.

3. Design

3.1 Design Objective

The design phase defines how the AI-powered employee productivity analytics and burnout prediction system will work end to end. The goal is to create a secure, role-based, scalable platform that supports employee monitoring, manager oversight, HR analytics, and ML-based burnout prediction.

3.2 System Architecture

The system follows a layered architecture: - Frontend: Angular-based UI for login, dashboards, employee views, manager views, and HR views - Backend: FastAPI REST API for business logic, authentication, and data access - Database: Relational database for users, employees, departments, worklogs, and predictions - ML Layer: Python-based prediction engine for burnout analysis - Authentication: JWT-based role-aware access control

3.3 Core Roles and Responsibilities

- Employee: Logs in, views own data, and records worklogs
- Manager: Views reportees, monitors team activity, and reviews productivity
- HR: Views all employees, recent hires, and burnout insights
- Admin/System: Manages setup, users, roles, and configuration

3.4 Functional Design

The system is designed around these major functions: - User authentication and token-based session management - Role-based access to pages and APIs - Employee data management - Manager reportee tracking - HR analytics and recent-hire review - Worklog capture and monitoring - Burnout prediction using ML model output

3.5 Data Design

Key entities in the system include: - Users - Roles - Employees - Departments - WorkLogs - Predictions

Important relationships: - One user can be linked to one employee profile - One department can have many employees - One manager can have many reportees - One employee can have many worklogs and predictions

3.6 API Design

The backend is designed as REST APIs with clear responsibilities: - Authentication APIs for login and current-user lookup - Employee APIs for profile and listing access - Manager APIs for reportee data - HR APIs for recent hires and burnout summaries - Worklog APIs for create/list operations - Prediction APIs for burnout scoring

3.7 UI/UX Design

The frontend follows a login-first design: - Users see only the login page before authentication - After login, the UI changes based on role - Navigation and pages are shown only if the user has access - HR sees organization-level dashboards - Managers see their team information - Employees see only their own data

3.8 Security Design

Security is built into the design through: - JWT authentication - Role-based authorization - Protected API routes - Server-side validation - Limited access to sensitive employee data - Recommendation for secure token handling in production

3.9 ML Design

The ML layer is designed to: - Train on employee productivity and burnout-related features - Store reusable model artifacts - Load the trained model at runtime - Produce burnout predictions for analytics and decision support

3.10 Deployment Design

The deployment design separates concerns: - Angular frontend runs in the browser - FastAPI backend serves APIs - Database stores application records - ML artifacts are loaded by the backend - The system can be deployed locally for development or on a server for production

3.11 Design Outcome

The design phase results in: - A clear software architecture - A role-based access model - A normalized data model - Defined REST APIs - A responsive frontend structure - An ML-ready backend for burnout prediction

4. Coding & Implementation

4.1 Backend Implementation

The backend is implemented using FastAPI and SQLAlchemy. It provides a modular structure for authentication, employee management, manager views, HR analytics, worklogs, and machine learning integration.

Key Backend Modules

- `app/main.py` or equivalent startup module for application bootstrapping
- `app/models.py` for database entities
- `app/schemas.py` for request and response validation
- `app/api/auth.py` for login and user session handling
- `app/api/employees.py` for employee listing and profile access
- `app/api/managers.py` for manager reportees
- `app/api/hr.py` for HR analytics features
- `app/api/worklogs.py` for worklog create/list operations
- `app/ml/predict.py` for prediction loading and inference

Implementation Highlights

- SQLAlchemy ORM models define the database structure.
- Pydantic schemas validate API requests and responses.
- JWT-based authentication protects endpoints.
- Role-based logic limits access to HR, Manager, and Employee data.
- Prediction artifacts are loaded from trained model files.

4.2 Database Implementation

The database is designed with normalized tables to reduce redundancy and improve integrity. During development, SQLite is used for local storage. The

design also supports other SQL databases through SQLAlchemy connection strings.

Key Database Features

- User and role mapping
- Employee profile storage
- Department assignment
- Worklog storage
- Prediction records
- Manager relationships

4.3 ML Implementation

The machine learning part of the project uses Python and Scikit-Learn.

ML Process

- Synthetic training data generation
- Feature engineering
- Model training and validation
- Model serialization with Joblib
- Prediction loading in backend services

Prediction Outputs

- Burnout risk classification
- Productivity-related analytics

4.4 Frontend Implementation

The frontend is built using Angular standalone components.

Frontend Features

- Login-first user experience
- Role-aware navigation
- HR employee views
- Manager reportee views
- Worklog data entry
- Dashboard and analytics-ready layout

UI Data Flow

- User logs in through Angular UI
- Angular sends credentials to FastAPI backend
- Backend returns JWT access token

- Token is stored in session storage for the session
- API requests include the Authorization header

4.5 Seeding and Utilities

The project includes scripts for table creation, role seeding, employee seeding, and manager assignment. These utilities help populate the application with demo data for development and testing.

5. Testing

5.1 Testing Strategy

The system is validated through a combination of functional and integration testing approaches.

5.2 Types of Testing

- Unit Testing
- Integration Testing
- API Testing
- UI Testing
- Security Testing
- User Acceptance Testing

5.3 Functional Test Areas

- Login with valid and invalid credentials
- Role-based access to pages and APIs
- Employee list access
- Manager reportee access
- HR analytics access
- Worklog submission and retrieval
- Burnout prediction workflow

5.4 Sample Test Cases

Test Case	Expected Result
Valid login	User authenticated successfully
Invalid login	Authentication rejected
HR accesses employee list	Employee data returned
Manager views reportees	Only assigned reportees returned
Employee submits worklog	Worklog saved

Test Case	Expected Result
HR views recent hires	Grouped recent-hire data returned
Burnout prediction request	Prediction returned successfully

5.5 Quality Considerations

The project emphasizes correctness, validation, modular design, and clear error handling. Role-based restrictions and API validation reduce the risk of unauthorized access and malformed requests.

6. Application

This project is applicable in organizations that want to monitor employee productivity and detect burnout risk proactively.

6.1 Use Cases

- HR departments monitoring employee trends
- Managers reviewing team-level productivity
- Organizations tracking worklogs and recent hires
- Decision makers identifying employees at risk of burnout
- Development teams experimenting with analytics-driven HR systems

6.2 Business Value

- Improves visibility into employee workload patterns
- Supports timely HR intervention
- Helps managers make informed decisions
- Demonstrates integration of ML with real-world business software
- Reduces manual effort in monitoring employee performance

6.3 Real-World Benefits

- Better employee wellness monitoring
 - More informed leadership decisions
 - Structured employee data handling
 - Faster analytics through automated prediction
 - Improved operational planning
-

7. Conclusion

The AI-Powered Employee Productivity Analytics and Burnout Prediction System demonstrates how full-stack development and machine learning can be combined to solve a practical organizational problem. The system provides secure login, role-based access, employee tracking, manager and HR views, worklog handling, and burnout-related analytics.

The project successfully shows the use of Angular, FastAPI, SQLAlchemy, and Scikit-Learn in a single enterprise-style application. It also provides a strong foundation for future enhancements such as deeper analytics, improved dashboards, advanced route protection, and richer reporting.

8. Bibliography

APA Style References

- FastAPI. (2026). *FastAPI documentation*. <https://fastapi.tiangolo.com/>
 - SQLAlchemy. (2026). *SQLAlchemy ORM documentation*. <https://docs.sqlalchemy.org/>
 - Angular. (2026). *Angular documentation*. <https://angular.dev/>
 - Scikit-Learn. (2026). *Scikit-Learn user guide*. <https://scikit-learn.org/stable/>
 - Joblib. (2026). *Joblib documentation*. <https://joblib.readthedocs.io/>
 - Python Software Foundation. (2026). *Python documentation*. <https://docs.python.org/3/>
-

Appendix: Notes for Submission

You may replace the placeholders in the Title Page, Certificate, and Declaration sections with your personal details before final submission.

9. Detailed System Architecture

This section expands the architecture discussion so the report can be printed as a complete project document rather than a short summary. It is intentionally detailed so that the final PDF has sufficient academic depth.

9.1 Architectural Overview

The system uses a layered client-server model. Each layer handles one responsibility and communicates with the next through well-defined interfaces.

Layer 1: Presentation Layer

The presentation layer is built in Angular. It renders login pages, dashboards, employee lists, manager views, HR views, worklog forms, and analytical summaries. This layer is responsible for user interaction and display logic.

Layer 2: Application Layer

The application layer is implemented using FastAPI. It provides API endpoints, enforces security rules, transforms request data into service calls, and returns JSON responses.

Layer 3: Persistence Layer

The persistence layer is managed by SQLAlchemy ORM and the relational database. It stores structured records for users, employees, departments, worklogs, predictions, leaves, tasks, projects, meetings, notifications, and audit logs.

Layer 4: Intelligence Layer

The intelligence layer contains the machine learning prediction pipeline. It converts employee activity features into burnout scores and risk categories.

9.2 Communication Flow

The communication flow between the layers is as follows:

1. A user submits input through the Angular UI.
2. The frontend sends an HTTP request to the FastAPI backend.
3. FastAPI validates the request using schemas.
4. Business rules are applied by the service logic.
5. SQLAlchemy retrieves or stores records in the database.
6. If the request involves prediction, the ML module generates a risk output.
7. The backend returns a JSON response to Angular.
8. Angular updates the visible screen with the result.

9.3 Deployment View

The deployment view separates the browser, the backend server, and the database. This makes development and testing easier because the frontend can be restarted independently from the backend, and the database can be changed without redesigning the UI.

Suggested Deployment Diagram Description

- User Browser
- Angular Frontend
- FastAPI Application Server

- SQLite Development Database
- ML Model Artifact Store

Insert Screenshot 22: Deployment architecture diagram.

9.4 Security View

The security view focuses on authentication, authorization, and data access control. The current implementation uses JWT tokens and role-based route protection. Production environments should additionally support stronger token storage and secure transport.

9.5 Maintainability View

The project is maintainable because it separates concerns into modules:

- `app/api/` for route handlers,
- `app/models.py` for ORM tables,
- `app/schemas.py` for request and response validation,
- `app/services/` for business logic,
- `app/repositories/` for database access,
- `app/ml/` for prediction logic.

This organization reduces coupling and makes testing easier.

9.6 Scalability View

The system is scalable at the design level because the backend is API-driven. New frontend screens can be added without modifying the core server structure. Similarly, new database tables can be added using migrations when the project grows.

10. Module-Wise Explanation

10.1 Authentication Module

The authentication module handles registration, login, password verification, token creation, and current-user lookup.

Responsibilities

- Hash user passwords.
- Validate login credentials.
- Generate access tokens.
- Identify the current authenticated user.
- Enforce role-based access.

Sample Behavior

If a user enters invalid credentials, the backend returns **401 Unauthorized**. If the credentials are valid, the backend issues a bearer token that the frontend can reuse for API calls.

Insert Screenshot 23: Successful login response in Swagger or Postman.

10.2 Employee Module

The employee module provides employee registration, listing, and profile retrieval. HR users are the primary consumers of this module.

Key Functions

- Create employee profile.
- Retrieve employee list.
- View employee detail.
- Search by employee ID or code.

Data Fields

- Employee code
- First name
- Last name
- Gender
- Department
- Manager
- Joining date
- Experience
- Status

10.3 Manager Module

The manager module helps supervisors view the employees assigned to them. This supports team-level review without exposing unrelated employees.

Key Functions

- List reportees for a manager.
- Check team members by department.
- Review productivity and attendance trends.
- Support manager-level oversight.

Insert Screenshot 24: Manager reportee dashboard.

10.4 HR Module

The HR module provides organization-level visibility.

Key Functions

- List all employees.
- View recent hires.
- Review burnout summaries.
- Analyze department patterns.
- Support HR operations.

HR Value

This module is important because it gives HR a centralized way to review employee health indicators and overall organization structure.

10.5 Worklog Module

The worklog module records daily or periodic employee activity.

Fields in a Worklog Record

- Employee ID
- Date
- Working hours
- Meeting hours
- Task count
- Task completion percentage

Business Interpretation

Worklog records are useful because they reveal how time is spent across tasks and meetings. They also provide input features for burnout prediction.

10.6 Prediction Module

The prediction module receives input features and returns a burnout score and category.

Output Types

- Low risk
- Medium risk
- High risk

Interpretation

Prediction does not replace human judgement. It is used as a support mechanism to help HR and managers identify possible workload issues.

11. Database Tables and Data Dictionary

11.1 Users Table

Stores application login credentials and account metadata.

Column	Type	Description
id	Integer	Primary key
username	String	Unique login name
email	String	Unique email address
password_hash	String	Hashed password
employee_id	Integer	Linked employee profile
created_at	DateTime	Account creation time
updated_at	DateTime	Last update time

11.2 Roles Table

Stores role names such as Employee, Manager, and HR.

11.3 Employees Table

Stores personnel details and organization relationships.

Column	Type	Description
employee_code	String	Unique employee code
first_name	String	First name
last_name	String	Last name
manager_id	Integer	Self-referenced manager
department_id	Integer	Department relationship
joining_date	Date	Joining date
experience	Numeric	Experience in years
salary_grade	String	Salary category
status	String	Active/inactive state

11.4 WorkLogs Table

Stores daily productivity-related entries.

Column	Type	Description
employee_id	Integer	Related employee
date	Date	Log date
working_hours	Numeric	Total working time
meeting_hours	Numeric	Time spent in meetings

Column	Type	Description
task_count	Integer	Number of tasks
task_completion_percent	Numeric	Completion percentage

11.5 Predictions Table

Stores model output and forecast history.

Column	Type	Description
employee_id	Integer	Related employee
burnout_risk	String	Risk category
burnout_score	Float	Numeric score
productivity_score	Float	Productivity estimate
predicted_on	DateTime	Prediction timestamp

Insert Screenshot 25: Database table list or SQLite browser view.

12. API Documentation Summary

This section provides a more detailed explanation of the most important API endpoints.

12.1 Authentication APIs

POST /api/auth/register

Registers a new user and assigns a default role if needed.

POST /api/auth/login

Validates username and password and returns a bearer token.

GET /api/auth/me

Returns the profile information of the current authenticated user.

12.2 Employee APIs

GET /api/employees/

Returns a list of employees for authorized HR users.

GET /api/employees/{id}

Returns a single employee record by ID.

POST /api/employees/

Creates a new employee entry.

12.3 Prediction APIs

POST /api/predict/burnout

Accepts burnout-related feature values and returns a prediction response.

Example Request Body

```
{
  "employee_id": 10,
  "working_hours_per_day": 9,
  "overtime_hours": 3,
  "meeting_hours": 4,
  "leave_count": 1,
  "late_arrivals": 2,
  "task_completion_percent": 72,
  "performance_rating": 4.0
}
```

Example Response Body

```
{
  "employee_id": 10,
  "burnout_risk": "Medium",
  "burnout_score": 0.67
}
```

12.4 HR APIs

HR endpoints provide recent hire lists, manager burnout summaries, and employee views.

Insert Screenshot 26: HR API response in Swagger.

13. Detailed Testing Documentation

13.1 Authentication Test Cases

Test ID	Test Description	Input	Expected Output
T1	Valid login	Correct username and password	Access token
T2	Invalid login	Incorrect password	Error message
T3	Current user lookup	Valid token	User profile

13.2 Authorization Test Cases

Test ID	Test Description	Input	Expected Output
T4	HR employee list access	HR token	Employee list
T5	Unauthorized access	Employee token on HR route	Permission denied
T6	Manager reportee view	Manager token	Assigned reportees

13.3 Worklog Test Cases

Test ID	Test Description	Input	Expected Output
T7	Submit worklog	Valid worklog JSON	Record saved
T8	Empty worklog	Missing required values	Validation error

13.4 Prediction Test Cases

Test ID	Test Description	Input	Expected Output
T9	Burnout prediction	Feature payload	Risk and score
T10	Missing feature	Incomplete payload	Validation error

13.5 UI Test Cases

- Login form should submit credentials correctly.
- Navigation should update after login.
- Role-specific pages should load only when authorized.
- Logout should clear the session state.

Insert Screenshot 27: API testing checklist or passed test table.

14. Additional Implementation Notes

14.1 Data Seeding

The project includes scripts for creating tables, seeding roles and users, inserting employee data, assigning managers, and training the ML model.

Purpose of Seeding

- Provide demo data for evaluation.
- Enable testing without manual database entry.
- Support fast local setup.

14.2 Training Pipeline

The training scripts generate synthetic employee data, train models, and save model artifacts in the project model directory.

Training Workflow

1. Generate input features.
2. Engineer derived variables.
3. Split data for training and evaluation.
4. Train classification and regression models.
5. Evaluate performance.
6. Serialize artifacts with Joblib.

14.3 Runtime Prediction Flow

At runtime, the backend loads the model artifacts and performs inference when the prediction endpoint is called.

14.4 Screenshot Guide for Final Printout

To complete the report before PDF export, the following images should be captured manually:

1. Login screen.
2. HR employee list.
3. Manager reportee page.
4. Employee detail page.
5. Worklog submission page.
6. Prediction result page.
7. Swagger API docs.
8. Database table browser.
9. Terminal output showing backend startup.
10. Frontend running in the browser.

15. Extended Conclusion and Final Remarks

The project demonstrates a practical example of combining enterprise web development and machine learning. It is especially useful as an academic major project because it touches on many core concepts in the MCA curriculum, including software engineering, database design, backend development, frontend development, applied analytics, and software testing.

The solution can be expanded into a production-ready HR analytics system by adding stronger security controls, migration support, dashboards, and explainable AI features. Even in its current state, it provides a realistic basis for discussion, implementation, demonstration, and academic evaluation.

15.1 Final Benefit Summary

- Helps identify workload stress patterns.
- Supports managerial decision-making.
- Improves HR visibility.
- Demonstrates end-to-end application development.
- Serves as a strong base for future research.

15.2 Final Screenshot Placeholder

Insert Screenshot 28: Completed application overview screen.

Appendix: Extra Submission Checklist

Before printing the final PDF, confirm the following:

- All placeholders have been replaced with actual student and guide details.
- Screenshots have been inserted into every marked location.
- Page numbers are visible in the footer.
- The document uses consistent formatting and font size.
- The PDF preview has been checked for page breaks.
- The bibliography is complete.
- The report has been proofread for grammar and spelling.